WOXSEN UNIVERSITY

School of Technology | B.Tech Computer Science & Engineering
Full Stack Development Lab – DevOps

LAB ASSIGNMENT #23

Automated Log Rotation and Compression

A Bash script that rotates and compresses application logs older than 7 days, scheduled with cron

---

Submitted by	
Name	K Author Abhikhyath Gundi
Enrollment No.	24WU0101126
Programme / Section	B.Tech CSE – Panthers
Email	author.abhikhyath_2028@woxsen.edu.in
Domain	DevOps
Tools	Bash, cron (Ubuntu 26.04 on WSL2)
Repository	github.com/authorabhikhyath-blip/Lab-fsd-bash-cron
Date of submission	07 October 2026

Table of Contents

1. Assignment Number and Title 2
2. Objective 2
3. Background 2
3.1 What is log rotation? 2
3.2 Why rotate logs? 2
3.3 What is cron? 3
4. Tools Used 3
5. Project Structure 4
6. Design of the Script 4
6.1 Features 4
6.2 Algorithm 5
7. Steps Followed 5
Step 1 – Environment setup 5
Step 2 – Generate the sample log folder 6
Step 3 – Capture the BEFORE listing 7
Step 4 – Dry run 7
Step 5 – Run the rotation 7
Step 6 – Capture the AFTER listing and verify the archives 8
Step 7 – Help message and edge cases 8
8. Scheduling with cron 9
9. Results 10
9.1 Before-and-after measurements 10
9.2 Edge case test results 11
9.3 Deliverables checklist 11
10. Comparison with logrotate 11
11. Observations 12
12. Conclusion 13
13. References 13
Appendix A – rotate_logs.sh 14
Appendix B – create_test_logs.sh 16

(If the page numbers are not shown, right-click the table and choose "Update Field".)

1. Assignment Number and Title

Field	Details
Assignment No.	Lab #23
Domain	DevOps
Title	Write a Bash script that rotates and compresses app logs older than 7 days and schedule it with cron
Starter provided	Sample log folder
Tools	Bash, cron
Expected outcomes	Script, crontab entry, before/after folder listing

2. Objective

The objective of this lab is to automate a routine system administration task: keeping an application's log folder under control. Specifically, the work aims to:

1. Write a Bash script (`rotate_logs.sh`) that finds application `.log` files older than **7 days** and compresses them with gzip.
2. Delete compressed archives that are older than a retention period (**30 days**) so disk usage cannot grow forever.
3. Make the script safe and reusable: strict error handling, configurable settings, a dry-run mode, a help message and meaningful exit codes.
4. Schedule the script with **cron** so it runs every day at 2:00 AM without anyone running it by hand.
5. Prove that it works with before/after folder listings, size measurements, edge case tests and cron/syslog evidence.

3. Background

3.1 What is log rotation?

Applications write a continuous record of events, such as requests, errors and warnings, into log files. If nothing is done, these files grow without limit. Log rotation is the practice of regularly retiring old log files: they are closed, compressed and kept as archives for a while, and finally deleted when they are no longer needed. Plain-text logs are very repetitive, so compression usually saves 80–95% of their space.

3.2 Why rotate logs?

- **Disk space:** a full disk can crash the application, the database and even the operating system.

- **Performance:** small daily files are faster to search, open, copy and back up than one huge file.
- **Retention and compliance:** policies (and laws such as GDPR) say how long logs must, or may, be kept. Automatic deletion enforces this.
- **Analysis:** closed, dated archives are easy to send to a central log server or to keep as evidence.
- **Automation:** a scheduled job never forgets, unlike manual clean-up.

3.3 What is cron?

cron is the standard Unix/Linux time-based job scheduler. The cron daemon wakes up every minute, reads each user's crontab (cron table), and runs every command whose schedule matches the current time. A crontab line has five time fields followed by the command:

```
┌──────────── minute        (0-59)
│ ┌────────── hour          (0-23)
│ │ ┌──────── day of month  (1-31)
│ │ │ ┌────── month         (1-12)
│ │ │ │ ┌──── day of week   (0-7, 0 and 7 = Sunday)
│ │ │ │ │
0 2 * * *   /home/abhikhyath/lab23/rotate_logs.sh ...
```

`0 2 * * *` means minute 0 of hour 2, on every day of the month, every month and every day of the week. In other words, **every day at 02:00**. 2 AM was chosen because traffic is low then, so compressing files does not slow the application down. For testing, `*/2 * * * *` was used. The step value `*/2` means "every second minute".

4. Tools Used

Tool / Component	Version / Detail	Purpose in this lab
Windows 11 + WSL2	Kernel 6.18 (microsoft-standard-WSL2)	Runs a real Linux environment on the laptop
Ubuntu	26.04 LTS	Linux distribution where the lab was done
GNU Bash	5.3.9	Scripting language for both scripts
cron / crontab	Debian cron (systemd service)	Schedules the script daily at 02:00
find (GNU findutils)	-mtime, -print0	Selects files by age, handles spaces safely
gzip / zcat	gzip -9	Compresses logs; checks and reads archives
touch	touch -d "N days ago"	Backdates test files to simulate old logs
ls, du	ls -lh, du -sh	Before/after listings and size measurement
journalctl / rsyslog	systemd journal, /var/log/syslog	Evidence that cron executed the job
VS Code, Git, GitHub	–	Editing, version control, submission

5. Project Structure

The project is published at https://github.com/authorabhikhyath-blip/Lab-fsd-bash-cron. The repository has this layout:

```
Lab-fsd-bash-cron/
├── rotate_logs.sh         # main script: compress old logs, delete old archives
├── create_test_logs.sh    # builds the sample log folder (starter data)
├── LAB23_REPORT.md        # notes: setup, cron, edge cases, viva answers
└── logs/                  # sample log folder (generated)
    ├── app.log, error.log, access.log         -> fresh (0-5 days)
    ├── app-old.log, debug.log                 -> old (10-20 days)
    ├── payment service.log                    -> old, name contains a space
    ├── notes.txt                              -> not a .log file
    └── archive-recent.log.gz, archive-ancient.log.gz  -> 15 d and 45 d archives
```

The sample log folder was designed so that every rule of the script is tested at least once. The table below shows what each file is for:

File	Age	Expected result	Rule tested
app.log	0 days	Kept unchanged	Fresh logs are not touched
error.log	2 days	Kept unchanged	Fresh logs are not touched
access.log	5 days	Kept unchanged	Under the 7-day limit
app-old.log	10 days	→ app-old.log.gz	Compression of old logs
payment service.log	12 days	→ payment service.log.gz	Names with spaces
debug.log	20 days	→ debug.log.gz	Compression of old logs
notes.txt	15 days	Kept unchanged	Only *.log files are rotated
archive-recent.log.gz	15 days	Kept	Archive inside the 30-day retention
archive-ancient.log.gz	45 days	Deleted	Retention clean-up

6. Design of the Script

6.1 Features

Requirement	How rotate_logs.sh implements it
Strict mode	`set -euo pipefail`: stop on errors, on unset variables, and on failures inside pipes
Configuration	Options `-d/-a/-r` or env vars `LOG_DIR`, `DAYS`, `RETENTION_DAYS` (defaults ./logs, 7, 30)
Input validation	Folder must exist and be writable; day values must be whole numbers
Find old logs	`find DIR -maxdepth 1 -type f -name '*.log' -mtime +7 -print0`
Spaces in names	NUL-separated names read with `while IFS= read -r -d ''`; every variable quoted; `--` before names

Requirement	How rotate_logs.sh implements it
Compression	`gzip -9 -c` into `file.log.gz`, keeps the original timestamp (`touch -r`), then removes the .log
Retention	Deletes `*.gz` older than 30 days. This runs before compression
No overwrite	If an archive with the same name exists, a timestamp is added to the new name
Dry run	`-n / --dry-run` prints every action but changes nothing
Help	`-h / --help` prints usage, options and exit codes
Logging	Every line has a `[YYYY-MM-DD HH:MM:SS]` timestamp; errors go to stderr
Exit codes	0 success, 1 usage error, 2 folder missing/not writable, 3 compression failure

6.2 Algorithm

1. Read the defaults from environment variables, then override them with any command-line options.
2. Check the inputs: numbers must be valid, and the folder must exist and be writable. If not, exit with code 1 or 2.
3. Print a START line showing the settings in use.
4. Step 1 – Retention: find `*.gz` files older than RETENTION_DAYS and delete them (or only print them in dry-run mode).
5. Step 2 – Rotation: find `*.log` files older than DAYS. For each one, compress it to a .gz file, copy the original timestamp onto it, and remove the original. If anything fails, remove the partial archive and count the failure.
6. Print a DONE summary (compressed / deleted / failed) and exit with 0, or with 3 if any file failed.

Why delete before compressing? A new archive keeps the original log's age. If deletion ran second, a 40-day-old log would be compressed and then deleted in the same run, before anyone could use it. Running retention first prevents this.

The complete, commented source code of both scripts is given in Appendix A and Appendix B.

7. Steps Followed

Step 1 – Environment setup

WSL2 with Ubuntu 26.04 was used on Windows 11. cron and rsyslog were installed and the cron service was checked:

```
sudo apt update && sudo apt install -y cron rsyslog
sudo systemctl enable --now cron      # on WSL without systemd: sudo service cron start
mkdir -p ~/lab23 && cd ~/lab23
cp /mnt/c/Users/autho/'LAB - fsd'/lab23/*.sh .
sed -i 's/\r$//' *.sh && chmod +x *.sh   # remove Windows line endings, make executable
```

```
abhikhyath@lenovo-LOQ11: ~/lab23

abhikhyath@lenovo-LOQ11:~/lab23$ uname -sr
Linux 6.18.33.2-microsoft-standard-WSL2
abhikhyath@lenovo-LOQ11:~/lab23$ bash --version | head -1
GNU bash, version 5.3.9(1)-release (x86_64-pc-linux-gnu)
abhikhyath@lenovo-LOQ11:~/lab23$ systemctl is-active cron
active
abhikhyath@lenovo-LOQ11:~/lab23$ systemctl status cron --no-pager | head -4
● cron.service - Regular background program processing daemon
     Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-10-07 16:35:18 UTC; 19s ago
 Invocation: c01f4e3edb3747a6a42a0796b01c9024
```

Figure 1: *WSL2 Ubuntu environment – Bash 5.3 and the cron service active (running)*

Step 2 – Generate the sample log folder

`create_test_logs.sh` creates the nine files described in Section 5. It backdates them with `touch -d "N days ago"` to simulate logs of different ages.

```
abhikhyath@lenovo-LOQ11: ~/lab23

abhikhyath@lenovo-LOQ11:~/lab23$ ./create_test_logs.sh
created: ./logs/app.log  (age 0d, 50KB)
created: ./logs/error.log  (age 2d, 20KB)
created: ./logs/access.log  (age 5d, 80KB)
created: ./logs/app-old.log  (age 10d, 120KB)
created: ./logs/payment service.log  (age 12d, 60KB)
created: ./logs/debug.log  (age 20d, 200KB)
created: ./logs/notes.txt  (age 15d, 5KB)
created: ./logs/archive-recent.log  (age 15d, 30KB)
created: ./logs/archive-ancient.log  (age 45d, 30KB)

Test data ready in ./logs:
total 548K
-rw-r--r-- 1 abhikhyath abhikhyath  80K 2026-10-02 16:35 access.log
-rw-r--r-- 1 abhikhyath abhikhyath 120K 2026-09-27 16:35 app-old.log
-rw-r--r-- 1 abhikhyath abhikhyath  50K 2026-10-07 16:35 app.log
-rw-r--r-- 1 abhikhyath abhikhyath  225 2026-08-23 16:35 archive-ancient.log.gz
-rw-r--r-- 1 abhikhyath abhikhyath  224 2026-09-22 16:35 archive-recent.log.gz
-rw-r--r-- 1 abhikhyath abhikhyath 200K 2026-09-17 16:35 debug.log
-rw-r--r-- 1 abhikhyath abhikhyath  20K 2026-10-05 16:35 error.log
-rw-r--r-- 1 abhikhyath abhikhyath 5.0K 2026-09-22 16:35 notes.txt
-rw-r--r-- 1 abhikhyath abhikhyath  60K 2026-09-25 16:35 payment service.log
```

Figure 2: *Sample log folder created by create_test_logs.sh with backdated timestamps*

Step 3 – Capture the BEFORE listing

```
abhikhyath@lenovo-LOQ11: ~/lab23

abhikhyath@lenovo-LOQ11:~/lab23$ ls -lh --time-style=long-iso logs
total 548K
-rw-r--r-- 1 abhikhyath abhikhyath  80K 2026-10-02 16:35 access.log
-rw-r--r-- 1 abhikhyath abhikhyath 120K 2026-09-27 16:35 app-old.log
-rw-r--r-- 1 abhikhyath abhikhyath  50K 2026-10-07 16:35 app.log
-rw-r--r-- 1 abhikhyath abhikhyath  225 2026-08-23 16:35 archive-ancient.log.gz
-rw-r--r-- 1 abhikhyath abhikhyath  224 2026-09-22 16:35 archive-recent.log.gz
-rw-r--r-- 1 abhikhyath abhikhyath 200K 2026-09-17 16:35 debug.log
-rw-r--r-- 1 abhikhyath abhikhyath  20K 2026-10-05 16:35 error.log
-rw-r--r-- 1 abhikhyath abhikhyath 5.0K 2026-09-22 16:35 notes.txt
-rw-r--r-- 1 abhikhyath abhikhyath  60K 2026-09-25 16:35 payment service.log
abhikhyath@lenovo-LOQ11:~/lab23$ du -sh logs
552K    logs
```

Figure 3: *BEFORE: folder listing – six .log files (three older than 7 days) and two archives; 552 KB in total*

Step 4 – Dry run

Before changing anything, the script was run with `--dry-run` to preview its actions. The folder still contained 9 files afterwards, so nothing was modified.

```
abhikhyath@lenovo-LOQ11: ~/lab23

abhikhyath@lenovo-LOQ11:~/lab23$ ./rotate_logs.sh --dry-run
[2026-10-07 16:35:38] START dir=./logs age>7d retention>30d dry_run=1
[2026-10-07 16:35:38] DELETE   ./logs/archive-ancient.log.gz
[2026-10-07 16:35:38] DRY-RUN: rm -f -- ./logs/archive-ancient.log.gz
[2026-10-07 16:35:38] COMPRESS ./logs/payment service.log -> ./logs/payment service.log.gz
[2026-10-07 16:35:38] DRY-RUN: gzip -9 -c -- './logs/payment service.log' > './logs/payment service.log.gz' && rm -- './logs/payment service
.log'
[2026-10-07 16:35:38] COMPRESS ./logs/app-old.log -> ./logs/app-old.log.gz
[2026-10-07 16:35:38] DRY-RUN: gzip -9 -c -- './logs/app-old.log' > './logs/app-old.log.gz' && rm -- './logs/app-old.log'
[2026-10-07 16:35:38] COMPRESS ./logs/debug.log -> ./logs/debug.log.gz
[2026-10-07 16:35:38] DRY-RUN: gzip -9 -c -- './logs/debug.log' > './logs/debug.log.gz' && rm -- './logs/debug.log'
[2026-10-07 16:35:38] DONE compressed=3 deleted=1 failed=0
abhikhyath@lenovo-LOQ11:~/lab23$ ls logs | wc -l
9
```

Figure 4: *Dry run: planned actions are printed (yellow) and no files are changed*

Step 5 – Run the rotation

```
abhikhyath@lenovo-LOQ11: ~/lab23

abhikhyath@lenovo-LOQ11:~/lab23$ ./rotate_logs.sh -d ./logs -a 7 -r 30; echo "exit code: $?"
[2026-10-07 16:35:38] START dir=./logs age>7d retention>30d dry_run=0
[2026-10-07 16:35:38] DELETE   ./logs/archive-ancient.log.gz
[2026-10-07 16:35:38] COMPRESS ./logs/payment service.log -> ./logs/payment service.log.gz
[2026-10-07 16:35:38] COMPRESS ./logs/app-old.log -> ./logs/app-old.log.gz
[2026-10-07 16:35:38] COMPRESS ./logs/debug.log -> ./logs/debug.log.gz
[2026-10-07 16:35:38] DONE compressed=3 deleted=1 failed=0
exit code: 0
```

Figure 5: *Real run: 3 logs compressed, 1 expired archive deleted, exit code 0*

Step 6 – Capture the AFTER listing and verify the archives

```
abhikhyath@lenovo-LOQ11: ~/lab23

abhikhyath@lenovo-LOQ11:~/lab23$ ls -lh --time-style=long-iso logs
total 176K
-rw-r--r-- 1 abhikhyath abhikhyath  80K 2026-10-02 16:35 access.log
-rw-r--r-- 1 abhikhyath abhikhyath  530 2026-09-27 16:35 app-old.log.gz
-rw-r--r-- 1 abhikhyath abhikhyath  50K 2026-10-07 16:35 app.log
-rw-r--r-- 1 abhikhyath abhikhyath  224 2026-09-22 16:35 archive-recent.log.gz
-rw-r--r-- 1 abhikhyath abhikhyath  806 2026-09-17 16:35 debug.log.gz
-rw-r--r-- 1 abhikhyath abhikhyath  20K 2026-10-05 16:35 error.log
-rw-r--r-- 1 abhikhyath abhikhyath 5.0K 2026-09-22 16:35 notes.txt
-rw-r--r-- 1 abhikhyath abhikhyath  329 2026-09-25 16:35 payment service.log.gz
abhikhyath@lenovo-LOQ11:~/lab23$ du -sh logs
180K    logs
abhikhyath@lenovo-LOQ11:~/lab23$ gzip -t logs/*.gz && echo 'all archives OK'
all archives OK
abhikhyath@lenovo-LOQ11:~/lab23$ zcat 'logs/payment service.log.gz' | head -2
2026-10-07 16:35:38 INFO app started request id=12345 status=200
2026-10-07 16:35:38 INFO app started request id=12345 status=200
```

Figure 6: *AFTER: old logs are now .gz archives, fresh logs untouched; folder reduced to 180 KB; archives pass gzip -t*

Step 7 – Help message and edge cases

```
abhikhyath@lenovo-LOQ11: ~/lab23

abhikhyath@lenovo-LOQ11:~/lab23$ ./rotate_logs.sh --help
Usage: rotate_logs.sh [-d DIR] [-a DAYS] [-r RETENTION_DAYS] [-n|--dry-run] [-h|--help]

  -d, --dir DIR          log folder              (env LOG_DIR, default ./logs)
  -a, --age DAYS         gzip .log older than    (env DAYS, default 7)
  -r, --retention DAYS   delete .gz older than   (env RETENTION_DAYS, default 30)
  -n, --dry-run          show actions, change nothing
  -h, --help             show this help

Example: rotate_logs.sh -d /var/log/myapp -a 7 -r 30
Exit codes: 0 ok, 1 usage error, 2 folder missing, 3 compression failure
```

Figure 7: *Usage/help message of rotate_logs.sh*

```
abhikhyath@lenovo-LOQ11: ~/lab23

abhikhyath@lenovo-LOQ11:~/lab23$ ./rotate_logs.sh -d nope; echo "exit code: $?"
[2026-10-07 16:35:38] ERROR: log directory 'nope' does not exist
exit code: 2
abhikhyath@lenovo-LOQ11:~/lab23$ ./rotate_logs.sh -d empty; echo "exit code: $?"
[2026-10-07 16:35:38] START dir=empty age>7d retention>30d dry_run=0
[2026-10-07 16:35:38] DONE compressed=0 deleted=0 failed=0
exit code: 0
abhikhyath@lenovo-LOQ11:~/lab23$ ./rotate_logs.sh; echo "exit code: $?"   # second run
[2026-10-07 16:35:38] START dir=./logs age>7d retention>30d dry_run=0
[2026-10-07 16:35:38] DONE compressed=0 deleted=0 failed=0
exit code: 0
abhikhyath@lenovo-LOQ11:~/lab23$ ./rotate_logs.sh -a x; echo "exit code: $?"
[2026-10-07 16:35:38] ERROR: DAYS must be a non-negative integer (got 'x')
exit code: 1
```

Figure 8: *Edge cases: missing folder (exit 2), empty folder (exit 0), second run (nothing to do), invalid value (exit 1)*

8. Scheduling with cron

The crontab was edited with `crontab -e`. Two entries were added: the permanent daily job, and a temporary job every 2 minutes to prove that cron really runs the script. Absolute paths are used because cron starts jobs in `$HOME` with a very small PATH. `>> rotate_cron.log 2>&1` appends both normal output and errors to a log file, because cron has no terminal.

```
# m h dom mon dow   command
0 2 * * *   /home/abhikhyath/lab23/rotate_logs.sh -d /home/abhikhyath/lab23/logs -a 7 -r 30 \
            >> /home/abhikhyath/lab23/rotate_cron.log 2>&1
*/2 * * * * /home/abhikhyath/lab23/rotate_logs.sh -d /home/abhikhyath/lab23/logs \
            >> /home/abhikhyath/lab23/rotate_cron.log 2>&1      # temporary test entry
```

(In the real crontab each entry is on one line. The lines are wrapped here only to fit the page.)

```
abhikhyath@lenovo-LOQ11: ~/lab23

abhikhyath@lenovo-LOQ11:~/lab23$ crontab -l
0 2 * * * /home/abhikhyath/lab23/rotate_logs.sh -d /home/abhikhyath/lab23/logs -a 7 -r 30 >> /home/abhikhyath/lab23/rotate_cron.log 2>&1
*/2 * * * * /home/abhikhyath/lab23/rotate_logs.sh -d /home/abhikhyath/lab23/logs >> /home/abhikhyath/lab23/rotate_cron.log 2>&1
abhikhyath@lenovo-LOQ11:~/lab23$ date
Wed Oct  7 16:35:38 UTC 2026
```

Figure 9: *crontab -l showing the daily 02:00 entry and the temporary every-2-minutes entry*

Fresh test data was generated again, and cron was left to run on its own. The output log and the system journal both show the script being run by cron at the scheduled minutes:

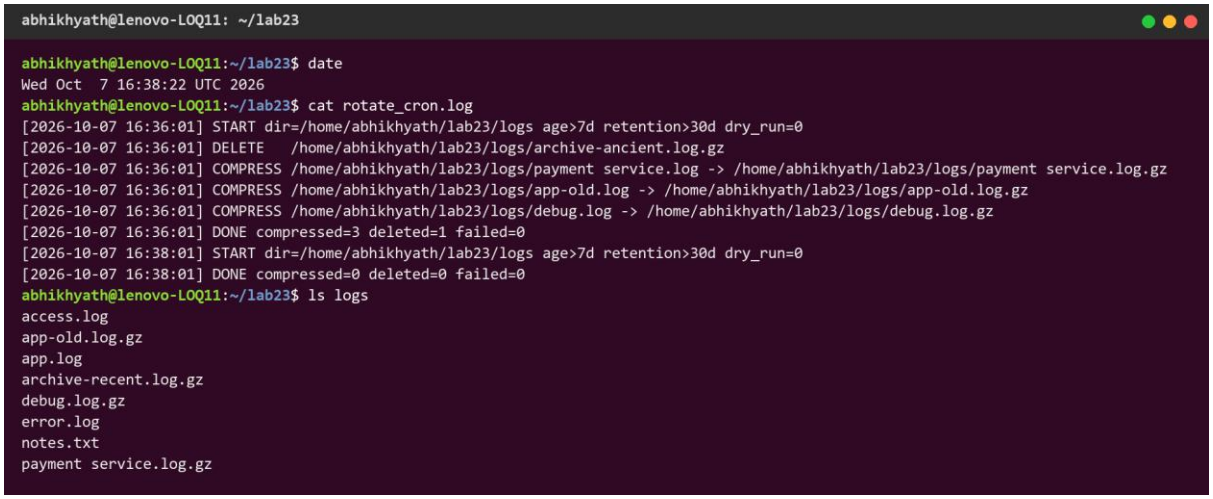


Figure 10: *rotate_cron.log written by cron-triggered runs (first run rotates, later runs find nothing to do)*



Figure 11: *System log entries proving cron executed the command as user abhikhyath*

After this was confirmed, the temporary entry was removed, leaving only the daily job:

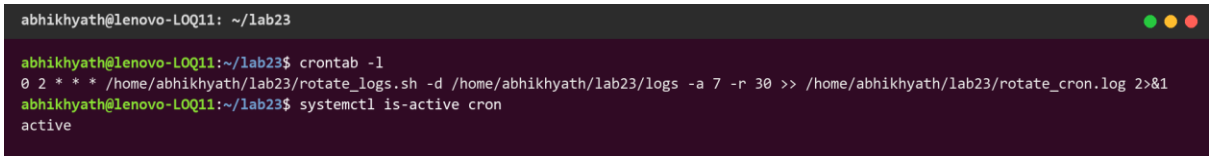


Figure 12: *Final crontab – only the daily 02:00 rotation job remains*

9. Results

9.1 Before-and-after measurements

Measurement	Before	After	Change
Total folder size (du -sh)	552 KB	180 KB	−372 KB (−67%)
Number of files	9	8	−1 (expired archive deleted)
.log files	6	3	3 rotated
.gz archives	2	4	+3 new, −1 expired
Size of the 3 old logs	380 KB	1.6 KB (530 + 806 + 329 B)	≈ 99.6% smaller
Fresh logs (app, error, access)	150 KB	150 KB	unchanged (correct)

File	Before	After
app-old.log (10 d)	120 KB .log	app-old.log.gz – 530 B

File	Before	After
debug.log (20 d)	200 KB .log	debug.log.gz – 806 B
payment service.log (12 d)	60 KB .log	payment service.log.gz – 329 B
archive-ancient.log.gz (45 d)	225 B	deleted
archive-recent.log.gz (15 d)	224 B	224 B (kept)
notes.txt (15 d)	5.0 KB	5.0 KB (ignored – not .log)

The test logs repeat the same line many times, so they compress extremely well (about 99%). Real application logs are less repetitive and usually shrink by 80–95%, which is still a large saving.

9.2 Edge case test results

#	Test case	Expected	Actual	Exit
1	Empty folder	Runs, nothing to do	compressed=0 deleted=0	0 ✓
2	Missing folder	Clear error	ERROR: log directory 'nope' does not exist	2 ✓
3	Spaces in file name	Handled safely	payment service.log → .gz	0 ✓
4	Running twice	Second run changes nothing	compressed=0 deleted=0	0 ✓
5	Dry run	Only prints actions	9 files before and after	0 ✓
6	Invalid number (-a x)	Rejected	ERROR: DAYS must be a non-negative integer	1 ✓
7	Fresh logs / non-.log files	Untouched	app/error/access/notes kept	0 ✓
8	Scheduled by cron	Runs without a user	Entries in rotate_cron.log and journal	✓

9.3 Deliverables checklist

Expected outcome	Delivered
Script	rotate_logs.sh (Appendix A) + create_test_logs.sh (Appendix B)
Crontab entry	0 2 * * * … rotate_logs.sh … >> rotate_cron.log 2>&1 (Figures 9 and 12)
Before/after folder listing	Figures 3 and 6, Table 9.1

10. Comparison with logrotate

Linux distributions include **logrotate**, a dedicated tool for the same job. The two approaches compare as follows:

Aspect	rotate_logs.sh + cron	logrotate
Installation	Nothing extra (bash, find, gzip)	Package logrotate (usually preinstalled)

Aspect	rotate_logs.sh + cron	logrotate
Trigger	Our own crontab entry	System daily timer / /etc/cron.daily
Rotation rule	File age (mtime)	daily / weekly / size; keeps a state file
Log files still being written	Not handled (only old, finished files)	copytruncate or postrotate to signal the app
History	Age-based deletion	Numbered copies (rotate N), dateext
Best for	Learning, simple age-based clean-up, custom logic	Production servers, standard practice

An equivalent logrotate configuration (`/etc/logrotate.d/myapp`) would be:

```
/home/abhikhyath/lab23/logs/*.log {
    daily
    rotate 30
    maxage 30
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
    dateext
    su abhikhyath abhikhyath
}
```

It can be tested with `sudo logrotate -d /etc/logrotate.d/myapp`, which is a debug/dry run.

11. Observations

- `find -mtime +7` counts whole 24-hour periods, so a file must be at least 8 full days old to match. A file that is 7.5 days old is not rotated yet.
- Without NUL-separated names (`-print0` / `read -d ''`), the file `payment service.log` would be split into two names and the script would fail. Quoting every variable is essential.
- With `set -o pipefail`, a pipeline like `yes | head` exits with code 141 (SIGPIPE). The test script uses process substitution instead.
- Under `set -e`, `((count++))` stops the script when count is 0, because the expression evaluates to 0 and returns status 1. `count=$((count+1))` is safe.
- Running retention before compression, and keeping the original timestamp with `touch -r`, makes the archive age reflect the log's real age.
- cron runs with a minimal environment (PATH=/usr/bin:/bin, no .bashrc). Absolute paths and output redirection are needed, or the job fails silently.
- Scripts copied from Windows have CRLF line endings, which cause `bad interpreter: /bin/bash^M`. `sed -i 's/\r$//'` fixes this.
- On WSL, cron only runs while the WSL virtual machine is running. On a real server the daily 02:00 job would run unattended.

- The script is idempotent: running it again immediately changes nothing. This makes it safe to schedule often.

12. Conclusion

The lab met its objective. A single, well-commented Bash script, `rotate_logs.sh`, finds application logs older than 7 days, compresses them with gzip and removes archives older than 30 days. Strict mode, input validation, safe file-name handling, a dry-run option, timestamped output and meaningful exit codes make it safe to run unattended. Scheduled with one crontab line, `0 2 * * *`, it runs every night at 2 AM. The 2-minute test entry, the output log and the system journal together proved that cron executed it.

The measurements show the benefit clearly: the sample folder shrank from 552 KB to 180 KB, and the three old logs went from 380 KB to about 1.6 KB, while recent logs stayed untouched for debugging. The lab also showed why logrotate is preferred on production servers: it handles logs that are still being written and keeps numbered history. Both tools follow the same principles practised here: rotate by age, compress, enforce retention, and automate with a scheduler.

13. References

- GNU Bash Reference Manual – https://www.gnu.org/software/bash/manual/
- crontab(5) and cron(8) manual pages – `man 5 crontab`
- GNU findutils manual (-mtime, -print0) – https://www.gnu.org/software/findutils/manual/
- gzip manual – https://www.gnu.org/software/gzip/manual/
- logrotate(8) manual page – `man logrotate`
- Project repository – https://github.com/authorabhikhyath-blip/Lab-fsd-bash-cron

Appendix A – rotate_logs.sh

```
#!/usr/bin/env bash
# ================================================================================
# rotate_logs.sh - compress .log files older than DAYS and delete .gz archives
#                  older than RETENTION_DAYS.                    (Lab #23)
#
# Exit codes: 0 = success, 1 = bad usage, 2 = log folder missing/unusable,
#             3 = one or more files failed to compress
# ================================================================================

set -euo pipefail          # -e: exit on any failing command
                           # -u: using an unset variable is an error
                           # -o pipefail: a pipe fails if ANY command in it fails
IFS=$'\n\t'                # word-split only on newline/tab, never on spaces

# ---- defaults: environment variable if set, otherwise the fixed value -------
LOG_DIR="${LOG_DIR:-./logs}"              # folder that holds the logs
DAYS="${DAYS:-7}"                         # compress .log files older than this
RETENTION_DAYS="${RETENTION_DAYS:-30}"    # delete .gz files older than this
DRY_RUN=0                                 # 1 = only print what would happen

SCRIPT_NAME="$(basename "$0")"            # used in the help text

# ---- helper functions -------------------------------------------------------
log() {                                   # timestamped message to stdout
    printf '[%s] %s\n' "$(date '+%Y-%m-%d %H:%M:%S')" "$*"
}
err() {                                   # timestamped message to stderr
    printf '[%s] ERROR: %s\n' "$(date '+%Y-%m-%d %H:%M:%S')" "$*" >&2
}
usage() {                                 # help text (heredoc keeps formatting)
    cat <<EOF
Usage: $SCRIPT_NAME [-d DIR] [-a DAYS] [-r RETENTION_DAYS] [-n|--dry-run] [-h|--help]

  -d, --dir DIR          log folder              (env LOG_DIR, default ./logs)
  -a, --age DAYS         gzip .log older than    (env DAYS, default 7)
  -r, --retention DAYS   delete .gz older than   (env RETENTION_DAYS, default 30)
  -n, --dry-run          show actions, change nothing
  -h, --help             show this help

Example: $SCRIPT_NAME -d /var/log/myapp -a 7 -r 30
Exit codes: 0 ok, 1 usage error, 2 folder missing, 3 compression failure
EOF
}

# ---- parse command-line arguments (they override env vars) ------------------
while [[ $# -gt 0 ]]; do                  # loop while arguments remain
    case "$1" in
        -d|--dir)        LOG_DIR="${2:?missing value for $1}"; shift 2 ;;
        -a|--age)        DAYS="${2:?missing value for $1}"; shift 2 ;;
        -r|--retention)  RETENTION_DAYS="${2:?missing value for $1}"; shift 2 ;;
        -n|--dry-run)    DRY_RUN=1; shift ;;
        -h|--help)       usage; exit 0 ;;
        *)               err "unknown option: $1"; usage >&2; exit 1 ;;
    esac
done

# ---- validate input ---------------------------------------------------------
for v in DAYS RETENTION_DAYS; do          # both must be whole numbers
    if ! [[ "${!v}" =~ ^[0-9]+$ ]]; then  # ${!v} = value of the variable named by v
        err "$v must be a non-negative integer (got '${!v}')"; exit 1
    fi
done
if [[ ! -d "$LOG_DIR" ]]; then            # folder must exist
    err "log directory '$LOG_DIR' does not exist"; exit 2
fi
if [[ ! -w "$LOG_DIR" ]]; then            # and we must be able to write in it
    err "log directory '$LOG_DIR' is not writable"; exit 2
fi

# run CMD...: execute the command, or only print it in dry-run mode
run() {
    local IFS=" "                         # join "$*" with spaces (global IFS is newline)
    if [[ $DRY_RUN -eq 1 ]]; then log "DRY-RUN: $*"; else "$@"; fi
}
```

```
log "START dir=$LOG_DIR age>${DAYS}d retention>${RETENTION_DAYS}d dry_run=$DRY_RUN"

compressed=0; deleted=0; failed=0         # counters for the summary
                                          # (x=$((x+1)) is used, not ((x++)), because
                                          #  ((x++)) returns status 1 when x=0 and set -e would exit)

# ---- step 1: delete archives older than RETENTION_DAYS -----------------------
# Done FIRST so that logs compressed in this same run are never deleted at once.
# -print0 + read -d '' passes names separated by NUL bytes -> safe for spaces/newlines.
while IFS= read -r -d '' gz; do
    log "DELETE   $gz"
    run rm -f -- "$gz"                    # -- ends options: a name like "-x.gz" is safe
    deleted=$((deleted + 1))
done < <(find "$LOG_DIR" -maxdepth 1 -type f -name '*.gz' -mtime +"$RETENTION_DAYS" -print0)

# ---- step 2: compress (rotate) .log files older than DAYS --------------------
# -mtime +7 = modified more than 7 full days (7*24h) ago.
while IFS= read -r -d '' f; do
    dest="$f.gz"                          # app.log -> app.log.gz
    if [[ -e "$dest" ]]; then             # never overwrite an existing archive
        dest="${f%.log}-$(date '+%Y%m%d%H%M%S').log.gz"
    fi
    log "COMPRESS $f -> $dest"
    if [[ $DRY_RUN -eq 1 ]]; then
        log "DRY-RUN: gzip -9 -c -- '$f' > '$dest' && rm -- '$f'"
        compressed=$((compressed + 1))    # counts "would compress" in dry-run
    elif gzip -9 -c -- "$f" > "$dest" && touch -r "$f" -- "$dest" && rm -f -- "$f"; then
        # gzip -9: best compression, -c: write to stdout (we pick the name)
        # touch -r: archive keeps the original log's age, so retention counts from it
        compressed=$((compressed + 1))
    else
        err "failed to compress '$f'"
        rm -f -- "$dest"                  # remove half-written archive, keep original
        failed=$((failed + 1))
    fi
done < <(find "$LOG_DIR" -maxdepth 1 -type f -name '*.log' -mtime +"$DAYS" -print0)

# ---- summary + exit code -----------------------------------------------------
log "DONE compressed=$compressed deleted=$deleted failed=$failed"
if [[ $failed -gt 0 ]]; then exit 3; fi
exit 0
```

Appendix B – create_test_logs.sh

```
#!/usr/bin/env bash
# create_test_logs.sh - builds a logs/ folder with fresh and old files for Lab #23
# Usage: ./create_test_logs.sh [target_dir]   (default: ./logs)

set -euo pipefail                     # stop on errors, unset variables and failed pipes

LOG_DIR="${1:-./logs}"                # first argument, or ./logs if none given

rm -rf -- "$LOG_DIR"                  # start clean so every run gives the same result
mkdir -p -- "$LOG_DIR"                # create the folder (-p: no error if parents are missing)

# make_log NAME DAYS_AGO SIZE_KB : creates a file of SIZE_KB of text and backdates its mtime
make_log() {
    local name="$1" days="$2" kb="$3"  # local = variables visible only inside this function
    local path="$LOG_DIR/$name"
    # yes prints a line forever; head -c cuts it to the size we want (text compresses well).
    # Process substitution <( ) is used instead of a pipe: with pipefail, "yes | head"
    # would fail with SIGPIPE (exit 141) when head stops reading.
    head -c "$((kb * 1024))" < <(yes "$(date '+%F %T') INFO app started request id=12345 status=200") >
"$path"
    touch -d "$days days ago" -- "$path"   # set modification time to N days in the past
    echo "created: $path  (age ${days}d, ${kb}KB)"
}

# ---- fresh files (must NOT be touched by rotate_logs.sh) ----
make_log "app.log"             0  50
make_log "error.log"           2  20
make_log "access.log"          5  80

# ---- old files (> 7 days, must be gzipped) ----
make_log "app-old.log"        10 120
make_log "payment service.log" 12  60   # name with a space: tests safe quoting
make_log "debug.log"          20 200

# ---- not a .log file: must be ignored even though it is old ----
make_log "notes.txt"          15   5

# ---- existing archives: one young (kept), one older than 30 days (deleted) ----
make_log "archive-recent.log"  15  30 && gzip -f -- "$LOG_DIR/archive-recent.log"
touch -d "15 days ago" -- "$LOG_DIR/archive-recent.log.gz"
make_log "archive-ancient.log" 45  30 && gzip -f -- "$LOG_DIR/archive-ancient.log"
touch -d "45 days ago" -- "$LOG_DIR/archive-ancient.log.gz"

echo
echo "Test data ready in $LOG_DIR:"
ls -lh --time-style=long-iso -- "$LOG_DIR"
```